TEK306: Sikkerhet i mobile applikasjoner

Cross-platform Mobile Apps & Security

Content

- Platform fragmentation
- Cross-platform development
- Security in cross-platform Apps
- Security in Cordova
- Scripting attacks
- Security controls

Platform fragmentation





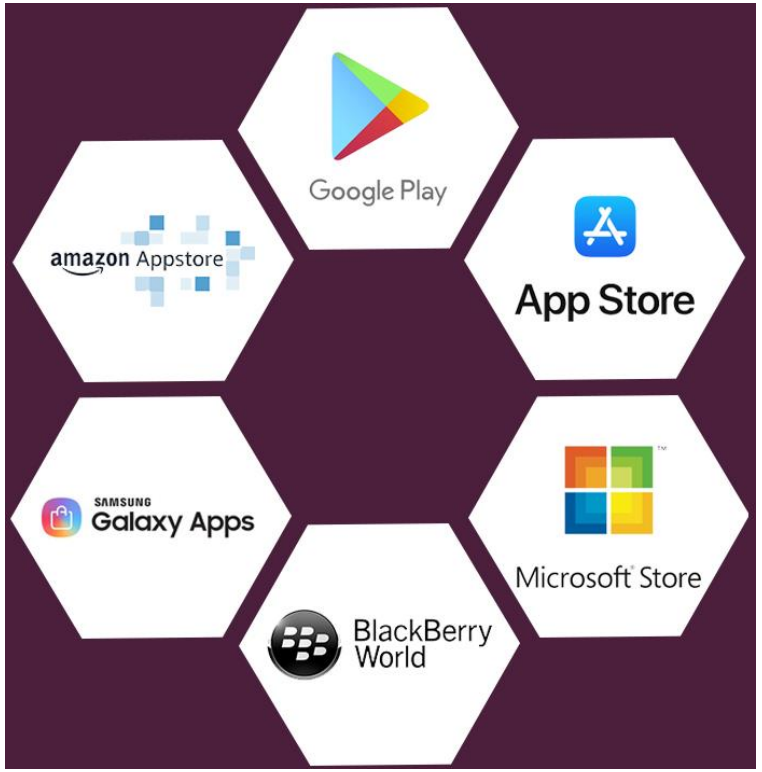
Multiple Teams/Products



Source: synerzip.com

Platform fragmentation …

- What are the **technologies** used to implement each of types of mobile apps?
- In what circumstances can one **choose to use** each type of mobile app?
- What are the **advantages/disadvantages** of each type of mobile app?



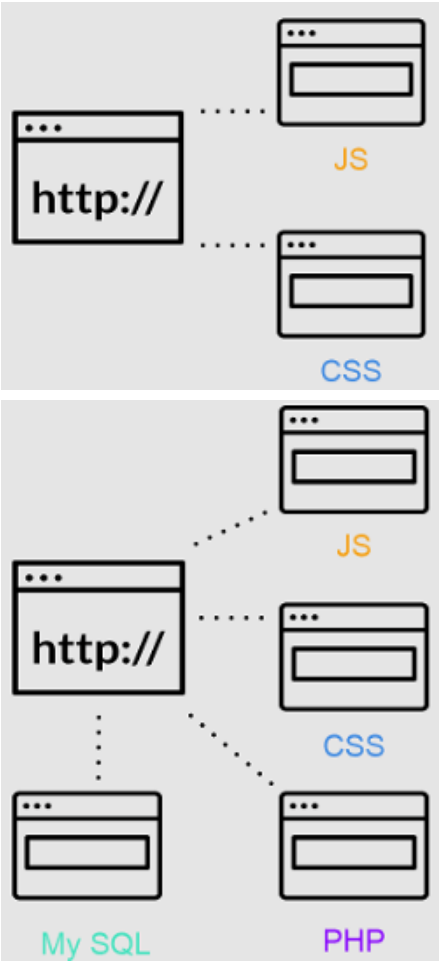
Platform fragmentation …



Platform fragmentation …

Web Apps

- Mobile Web App → mobile friendly Web
- Web App vs Website (vs Web page)
- **Web App**: every user **see different information** each time.
- Further reading: https://dinarys.com/blog/website-vs.-webapplication



Platform fragmentation …

Native vs Hybrid App Comparison

Features	Native app	Hybrid app
Language	Objective-C/Swift (iOS) Java or Kotlin (Android	HTML,CSS, Javascript
Performance	Faster and reliable	Slower than Native
Development Cost	Expensive	Moderate
Time of market	Slower	Fast
Speed	Fast	Slower than Native apps
User Experience	Rich and attractive	Never a fully native experience
Security	Secure	Less secure than native
Complexity	Complex coding	Write once, deploy everywhere kinda coding
Device feature integration	Seamless	Inadequate
Access to native APIs	High	Moderate
Frameworks	API provided by OS	React native PhoneGap Xamarin

Further readings on types of mobile apps:

- https://appinventiv.com/blog/native-vs-hybrid-app-development/
- https://www.mobiloud.com/blog/native-web-or-hybrid-apps#4
- https://steelkiwi.com/blog/how-choose-correct-platform-mobile-app-development/

Platform fragmentation …

Native Apps →

When to	When not to
High Performance Apps	Performance is not the main criteria
Heavy on OS and Device Features	Replicates Web Apps with few device features
Complex Network comm.	Standard RESTful
Canvas based Apps	Widget based apps
Only Few Platforms	Many platforms

Platform fragmentation …

Hybrid Apps →

When to	When not to
Performance is not the main criteria	High performance Apps
Replicates Web Apps with few device features	Heavy on OS and device features
Standard RESTful	Complex Network comm.
Widget based apps	Canvas based Apps
Many platforms	Only few platforms
Time to market is critical	
Saving cost is critical	

Cross-platform development

- **Web development** for common UI.
- **Native code** for heavy weight programming.
- Use frameworks like **Cordova plugin** development framework to **glue** the two.
- Use **automation** to regress every layer.



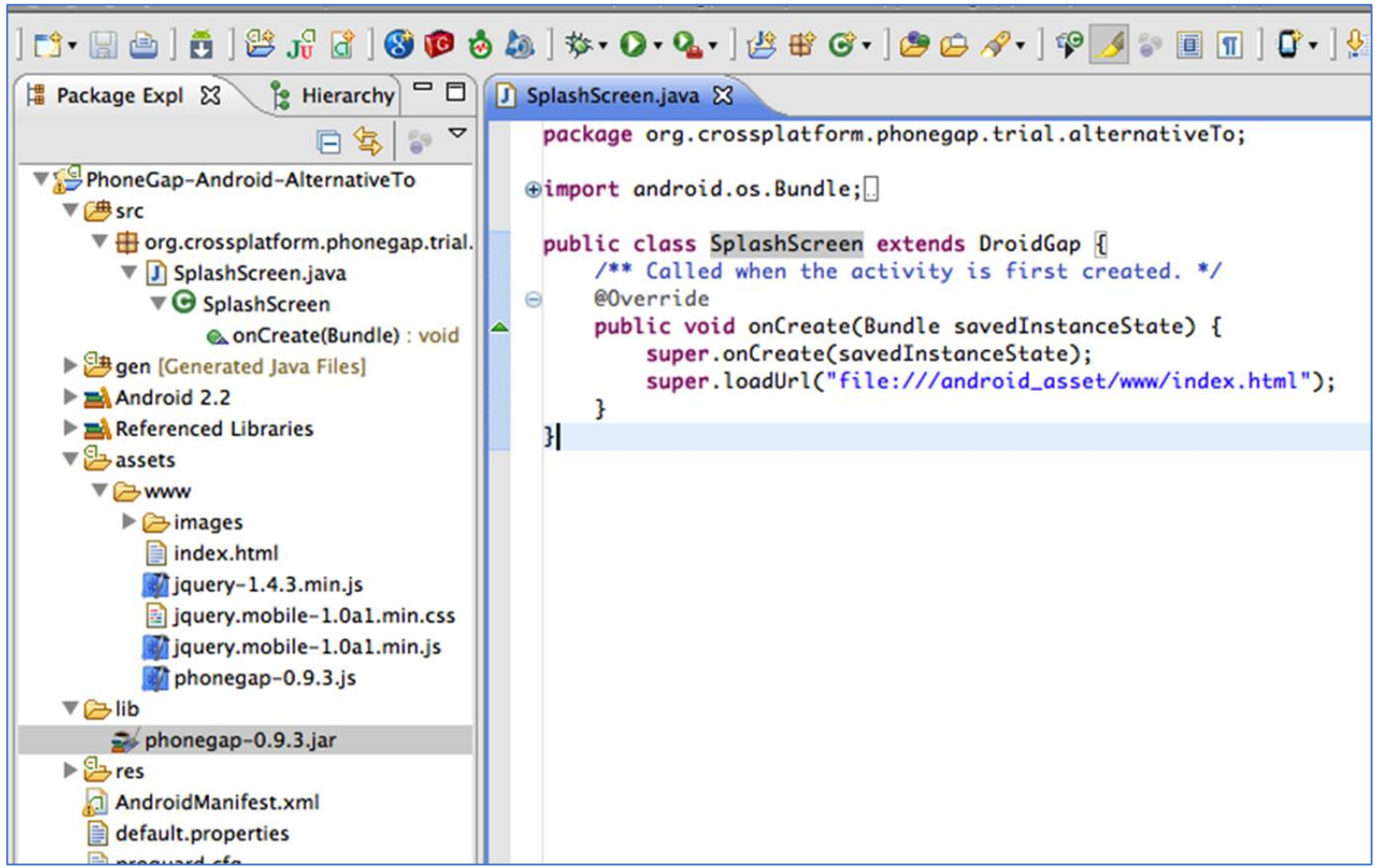
Cross-platform development …

Coding example in **Cordova**

- We extend **DroidGap** instead of an Activity.
- Internally, DroidGap uses **WebView** (show HTML, JavaScript & CSS like Web 2.0).
- WebView has **hooks to call Java from JavaScript** and Vice a Versa.
- WebView makes a call to JavaScript to allow **access to native mobile-resources** (e.g., GPS, Database, File, etc.)

Cross-platform development …

Coding example in **Cordova** …



Cross-platform development …

[Comparisons of frameworks]

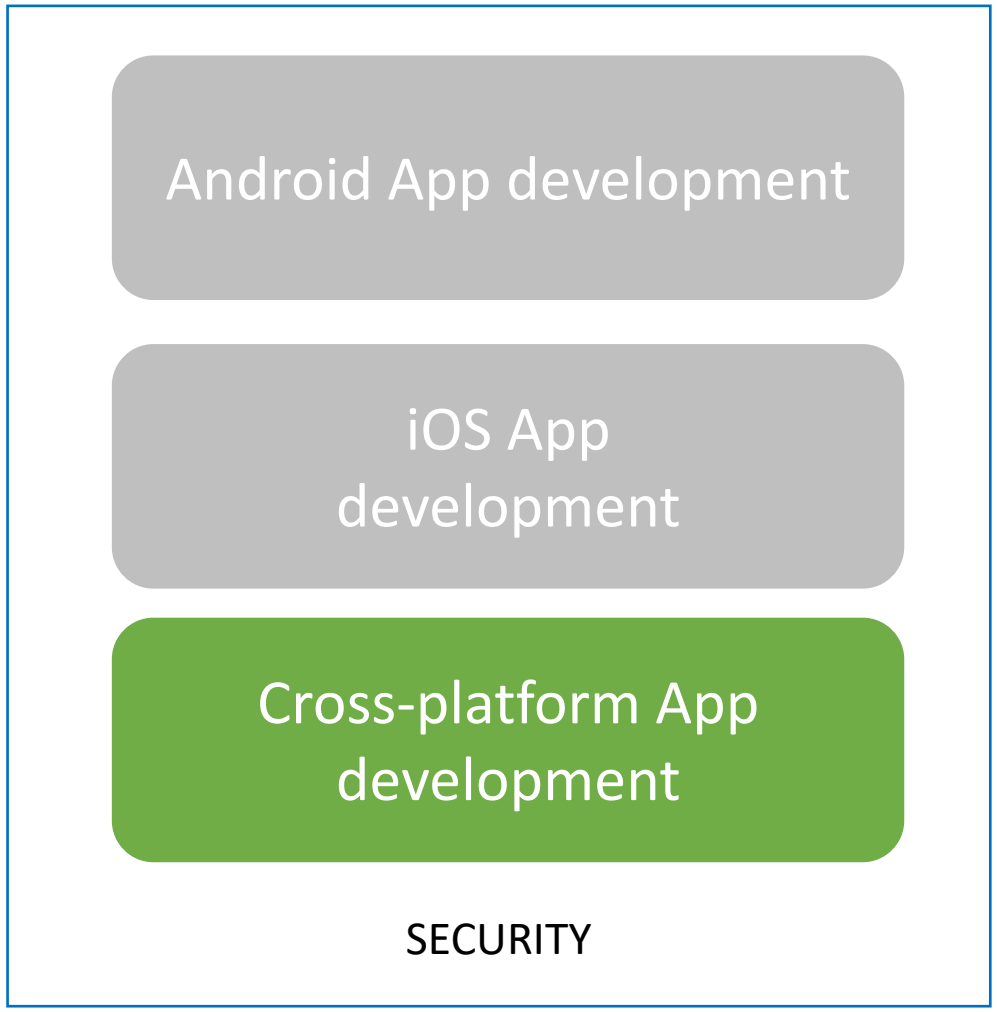
Attribute	ionic	React	Xamarin	Flutter
Programming Language	CSS, HTML5 and Typescript + JavaScript	Java, Swift, JavaScript+ or Objective C	C# with .net environment	Dart
Performance	Moderate ★★★★★	Close-to-native ★★★★★	iOS/Android: Close-to-native ★★★★★ Forms: Moderate ★★★★★	Amazing ★★★★★
User Interface	CSS, HTML	Uses Native UI Controllers	Uses Native UI Controllers	Uses Proprietary Widgets for stunning UI
Market & Community	Strong	Very Strong	Strong	New to market, so not very popular
Platforms Supported	Android 4.4+, iOS 8+, Windows 10	Android 4.1+, iOS 8+	Android 4.0.3+, iOS 8+, Window 10	Android Jelly Bean, v16, 4.1.x or newer and iOS 8+
Code Reusability	98% code reusable	90% code reusable	96% code reusable	50-90% code reusable
Well-known Application	JustWatch, Pacifica and Nationwide	Instagram, Facebook, Airbnb, UberEats	Storyo, Olo, The World Bank	GoogleAds, The New York Times, eBay
Costing	Open-source + Paid plans	Open-source	Open-source + Paid plans	Open-source

Cross-platform development …

What is the trend?

- **WebKit** as being the **next virtual machine**, which may go beyond the limits of Java.
- HTML5, CSS3 and JavaScript constitute the set of technology that **lessens the fragmentation**.
- HTML5 will **compete with native** components.

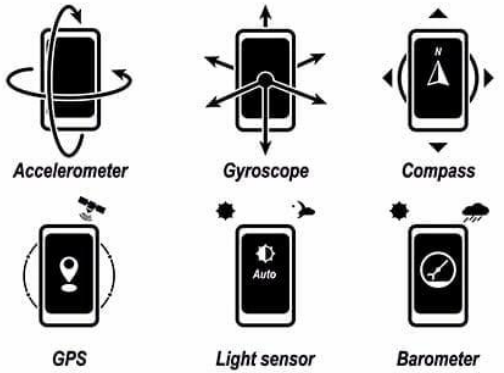
Security in cross-platform mobile apps



Security in Cordova

Features of Cordova API:

- Accelerometer- accesses device's motion sensor.
- Camera- captures photo using the device's camera.
- Compass- obtains direction of the device pointing.
- Contacts- works with the device's contact database.
- Filesystem- hooks into the device's file system.
- Geolocation- accesses the device's GPS location.
- Media- accesses (records) videos, audio or images.
- Network- performs accesses or requests network.
- Notifications- accesses visual device notifications.
- etc

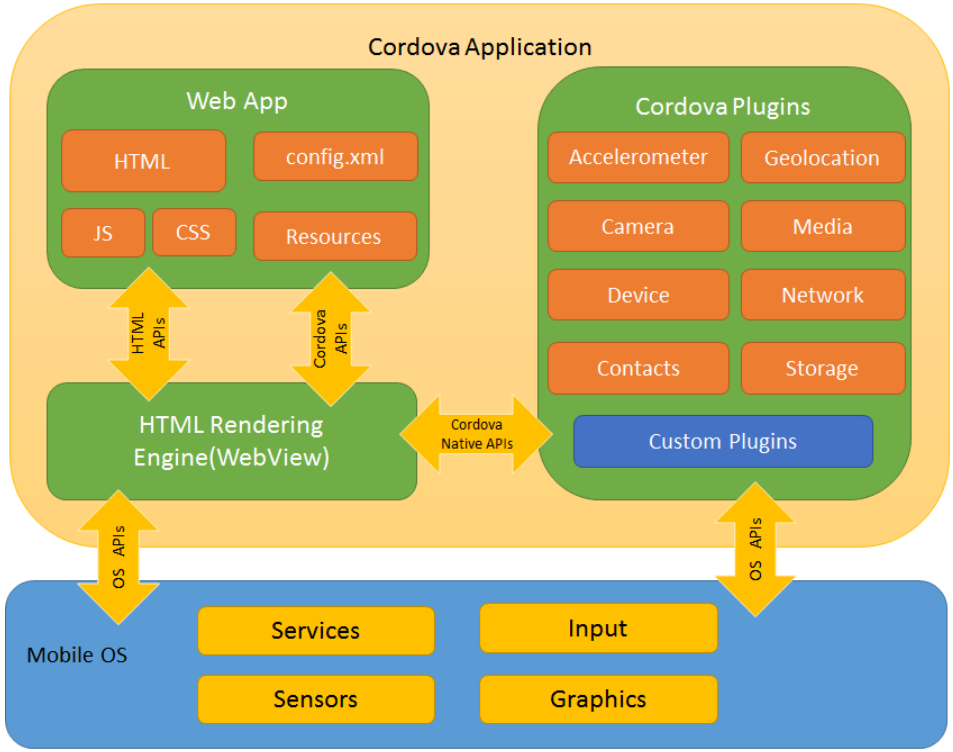



```
//Accessing camera using getPicture() API in Cordova
navigator.camera.getPicture(this.onPhotoDataSuccess,
this.onFail, {
        quality: 50,
        destinationType:
        Camera.DestinationType.DATA_URL,
        sourceType:
        Camera.PictureSourceType.CAMERA

});
```

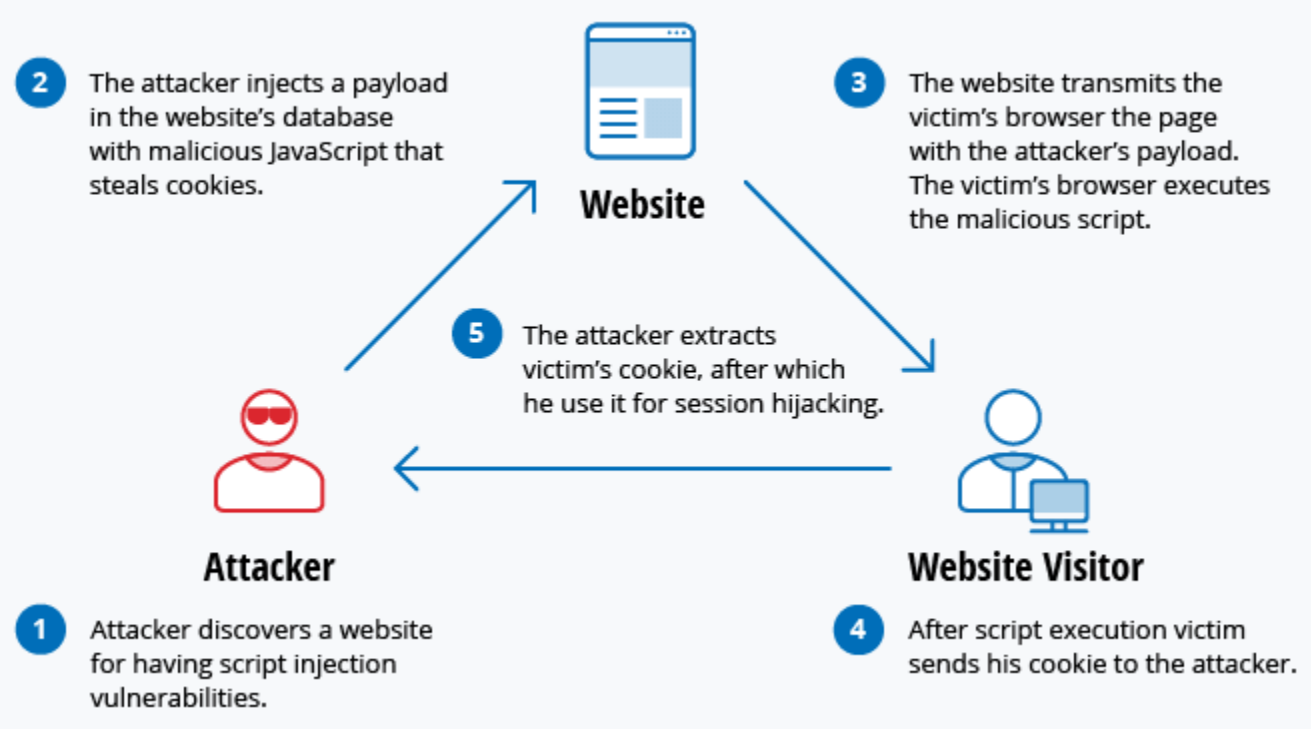
Security in Cordova …

- The Cordova frameworks did not come under the scrutiny of the **security community**.
- As the technology is a **blend of native and Web**, testing methods in both are applicable.
- However, a number of **framework-specific** security considerations are available.
- Cross-platform frameworks like Cordova depend on **WebView (the embedded browser)** availability in different platforms.



Scripting attacks

- Attacker-controlled data populated into WebView, causes **cross-site scripting or cross-application** attacks.



Scripting attacks …

Cross-site scripting (XSS)

- XSS is a type of security vulnerability typically found in **web apps.**
- It enable attackers to **inject client-side scripts** into **web pages** viewed by users.
- Such vulnerability is used by attackers to bypass access controls like **same-origin policy** (see Cross-site scripting).
- However, XSS not considered cross-application scripting …

Scripting attacks …

Cross-Application Scripting (CAS)

- CAS is a **vulnerability** affecting apps that don't check input in an exhaustive way.
- It allows an attacker to insert data that modifies the behavior of apps ...
- An example of CAS is when an **arbitrary URL is populated into Cordova WebView** when a third-party app invokes an intent.

Scripting attacks …

CAS attacks scenarios

- Tainted data from a **server-side Web App** (XSS) populated to the WebView.
- **Arbitrary URLs passed** through IPC mechanisms (CAS) are loaded.
- Data loaded via IPC into a WebView and dynamically populated **into a JavaScript block (CAS).**
- See link on how to **fix CAS** vulnerability

Security controls

Domain whitelisting

- Domain whitelisting is a **security control** in Cordova.
- It **defines external domains** that are from outside of the app's control- explicitly allowing to get access, privilege, service, mobility or recognition.
- Whitelisted domains have **access to the Cordova** JavaScript objects and corresponding Cordova bridge.
- It can be configured using the app's **config.xml** file as:

  <access origin="https://mobileapphacker.com" />.

- Example of insecure whitelist- allowing unrestricted access to any domai:

  <access origin="*"/>.

Security controls …

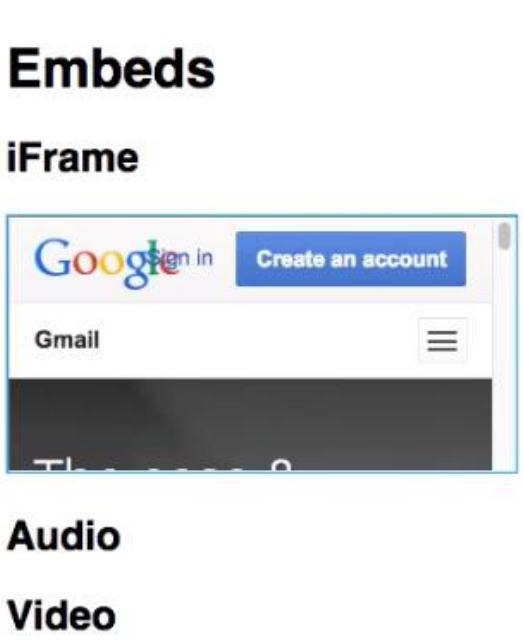
Iframes/Callbacks

- If a whitelisted domain **loads content via an iframe** the loaded content also has **access to the Cordova bridge**.
- Example of whitelisting an advertising network- **ads loaded by iframe** may accidentally expose Cordova bridge to any third-party sites.
- This means a **malicious ad could do any action** that the app itself could perform.
- There is one exception to this that when **Cordova is used on iOS**, all URLs are intercepted.

Security controls …

Iframes/Callbacks …

```
1  <!DOCTYPE html>
2  <html lang="">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport"
   content="width=device-width, initial-
   scale=1.0">
6    <link rel="stylesheet"
   href="css/style.css">
7    <title>Form</title>
8  </head>
9
10 <body>
11   <div class="wrap">
12     <h1>Embeds</h1>
13
14     <h2>iFrame</h2>
15     <iframe src="https://gmail.com"
   frameborder="2"></iframe>
16
17     <h2>Audio</h2>
18
19     <h2>Video</h2>
20
21   </div>
22 </body>
23 </html>
24
```



Security controls …

Encrypted Storage

- Cordova's file system APIs **do not support encryption**.
- Instead, **it relies** on the default behaviour of the platform.
- Cordova apps running on the **iOS platform inherit default protection**.
- However, encryption is not supported by default in other platforms- content may be stored **in plaintext**.
- Among the solutions to this could be using platform-specific methods like **Android keystore or iOS keychain**.

Security controls …

Security in other frameworks

- React native provides secure storage, authentication, etc.
- Xamarin's security:
  - **Transport Layer Security (TLS)** for- Android and iOS
  - Local Security Measures in Xamarin.Forms
  - Blocking COPY/PASTE feature from editable fields, blocking screen capture, etc.

Class activities:

1. Individual participation 1:
   Discussion questions via Mentimeter (Week45)
2. Individual participation 2:
   Questions via Kahoot (Week45)
3. Group activity:
   List of activities to be provided
4. Work on “Obligatory assignment 2“

NB: Additional information about the activities yet to be provided in separate document.

Next topic

- Course summary, wrap-up